1. JVM Memory Structure: Heap vs Stack

The JVM divides memory into two primary areas, each serving a distinct purpose for your application.

The Stack

- **Purpose:** Used for static memory allocation and **thread execution**. Each thread has its own Stack.
- **Content:** Stores primitive types (int, boolean, double) and **references** (memory addresses) to objects in the Heap.
- **Behavior:** Operates on a LIFO (Last-In-First-Out) basis. When a method is called, a "Frame" is pushed; when it returns, the frame is popped.
- **Error:** If the Stack runs out of space (usually due to infinite recursion), you get a StackOverflowError.

The Heap

- **Purpose:** Used for **dynamic memory allocation**. It is shared across all threads.
- **Content:** Stores all **Objects** (e.g., String, User, HashMap, ArrayList).
- **Behavior:** Objects stay here as long as they are referenced. Once no references exist, they are eligible for Garbage Collection.
- **Error:** If the Heap is full, you get an OutOfMemoryError.

---

2. Garbage Collection (GC) Basics

Garbage Collection is the process of identifying and deleting objects that are no longer reachable by the application.

The Generational Hypothesis

Most objects "die young." To optimize this, the Heap is divided into:

1. **Young Generation (Eden + Survivor Spaces):** Where new objects are created. Most are cleared quickly in a "Minor GC."
2. **Old Generation:** Where long-lived objects are moved after surviving multiple GC cycles. This is cleared in a "Major GC" (which is more expensive).

How it identifies "Garbage":

Java uses **Reachability Analysis**. It starts from "GC Roots" (like active threads or static variables) and follows the references. If an object cannot be reached through any path, it is marked for deletion.

---

3. The String Pool

The String Pool is a special storage area inside the Heap. Because String is the most used data type, Java uses this pool to save memory.

- **Literal vs New:** * String s1 = "Hello"; — Java checks the pool. If "Hello" exists, it reuses it. If not, it creates it in the pool.
    - String s2 = new String("Hello"); — This **forces** Java to create a brand new object in the general Heap, bypassing the efficiency of the pool.
- **Interning:** You can manually move a string into the pool using s2.intern().

---

4. String Immutability

In Java, once a String object is created, its value cannot be changed.

Why is it Immutable?

1. **Security:** Strings are used for sensitive data (usernames, passwords, connection URLs). If they were mutable, a malicious thread could change the URL *after* security checks were performed.
2. **Synchronization:** Being immutable makes Strings inherently **thread-safe**. Multiple threads can share a string without fear of one thread changing the value for others.
3. **Caching:** It allows the String Pool to exist. If a String could change, every reference to "Hello" would change if one person modified it.
4. **HashCode Caching:** Since the value doesn't change, the hashCode is calculated once and cached. This makes Strings incredibly fast when used as keys in a HashMap.

---

5. Practical Scenario: Performance Tuning

The Interviewer asks: "You are processing 1 million logs in a loop and concatenating strings like log = log + newEntry;. Why is this bad?"

The Senior Answer:

"Because Strings are immutable, every time you use +, Java creates a **new** String object and copies the old one into it. This creates $1,000,000$ temporary objects in the Heap, triggering massive Garbage Collection. Instead, I would use **StringBuilder** (for single-threaded) or **StringBuffer** (if thread-safety is needed). StringBuilder uses a mutable internal array, meaning it modifies the same object in memory, which is significantly faster and memory-efficient."

Final Technical Check:

- **Heap vs Stack:** Do you understand that a local variable *reference* lives on the Stack, but the *object* it points to lives on the Heap?
- **GC:** Are you aware that System.gc() is just a "hint" and doesn't guarantee the GC will run immediately?